

Lecture 4: The Minimum Spanning Tree Problem

Date: August 12, 2026
Instructor: Aniket Basu Roy
Scribe: Sai Kadam, Bhaskar Verma

Overview. Every problem we study this semester will be treated in the same four steps: *formalize* the problem precisely, *design* an algorithm, *prove* the algorithm correct, and *analyze* its efficiency. Today we carry out all four steps for the MINIMUM SPANNING TREE (MST) problem, covering two greedy algorithms and their correctness.

1 The Minimum Spanning Tree Problem

Motivation. Consider having n computers that need to be connected via a LAN, where the weight of each edge represents the length of cable required to connect the corresponding pair of computers. We want to connect all computers using the minimum possible total length of cable, while ensuring that there is a path between every pair of computers. In other words, we want to find a **minimum spanning tree** of the network. The Minimum Spanning Tree (MST) problem is a classical **greedy optimization problem**, where we make locally optimal choices at each step with the goal of obtaining a globally optimal solution.

Definition 1 (Instance). An instance consists of a connected, undirected, weighted graph $G = (V, E)$, where V is a set of vertices and E is a set of edges, together with a weight function $w : E \rightarrow \mathbb{R}$ that assigns a weight $w(e)$ to each edge $e \in E$.

Definition 2 (Spanning Tree). A spanning tree T of a graph $G = (V, E)$ is a subset of edges $T \subseteq E$ such that T connects all the vertices in V and contains no cycle. Equivalently, a spanning tree of a graph with $|V|$ vertices contains exactly $|V| - 1$ edges.

Definition 3 (Minimum Spanning Tree). Given a weighted graph $G = (V, E)$, a spanning tree T is a minimum spanning tree (MST) if its total weight is minimum among all spanning trees of G . The total weight of a spanning tree T is defined as

$$w(T) = \sum_{e \in T} w(e).$$

Thus, an MST T^* satisfies

$$w(T^*) = \min_{T \in \mathcal{T}} \sum_{e \in T} w(e),$$

where $\mathcal{T}$ is the set of all spanning trees of G .

Worked example. Consider a edge weighted graph with $V = \{A, B, C\}$ and edges AB, AC , and BC , with weights $w(AB) = 1$, $w(AC) = 3$, and $w(BC) = 2$.

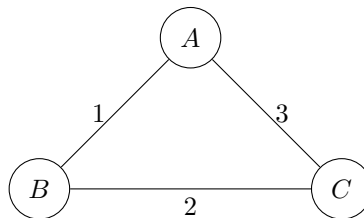


Figure 1: A weighted graph with three vertices.

The set of edges $\{AB, BC\}$ forms a spanning tree, since it connects all three vertices and contains no cycle. Its total weight is

$$w(T) = 1 + 2 = 3.$$

Another spanning tree, $\{AB, AC\}$ has total weight

$$w(T) = 1 + 3 = 4.$$

Since $\{AB, BC\}$ has the smaller total weight and we cannot do better than that, it is a minimum spanning tree of the graph. Thus, an arbitrary spanning tree need not be a minimum spanning tree, an algorithm is needed to select a spanning tree with minimum total weight.

2 Kruskal's Algorithm

Algorithm 1: Kruskal's Algorithm

Input: A connected, undirected, weighted graph $G = (V, E)$

Output: A minimum spanning tree T

```

1  $T \leftarrow \emptyset$ 
2 Sort all edges in  $E$  in non-decreasing order of their weights
3 foreach edge  $e = (u, v)$  in the sorted order do
4   if adding  $e$  to  $T$  does not create a cycle then
5      $\quad$  Add  $e$  to  $T$ 
6   if  $|T| = |V| - 1$  then
7      $\quad$  break
8 return  $T$ 
  
```

Algorithm 1 considers the edges in non-decreasing order of their weights and adds an edge to the growing spanning tree whenever doing so does not create a cycle. Thus, at each step, it makes the locally cheapest feasible choice. The algorithm terminates when $|V| - 1$ edges have been selected, since a spanning tree with $|V|$ vertices contains exactly $|V| - 1$ edges.

Worked example. Consider the weighted graph shown in Figure 2, with $V = \{A, B, C, D\}$ and edges AB, BC, AC, CD , with weights

$$w(AB) = 1, \quad w(BC) = 2, \quad w(AC) = 3, \quad w(CD) = 4.$$

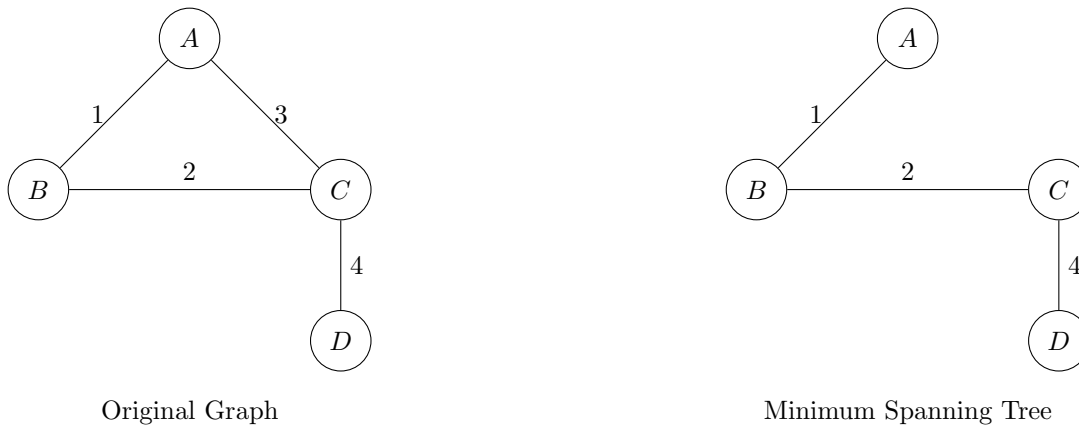


Figure 2: The input graph and the MST obtained by Kruskal's Algorithm.

The edges are considered in non-decreasing order of their weights: AB, BC, AC, CD . First, AB is added to T since it does not create a cycle. Next, BC is added for the same reason. The edge AC is then considered, but adding it would create the cycle $A - B - C - A$, so it is discarded. Finally, CD is added since it does not create a cycle. The algorithm then terminates because $|T| = |V| - 1 = 3$.

The output is $T = \{AB, BC, CD\}$ with total weight $w(T) = 1 + 2 + 4 = 7$. Thus, Kruskal's Algorithm returns the minimum spanning tree of the graph.

3 Prim’s-Jarník Algorithm

Algorithm 2: Prim’s-Jarník Algorithm

Input: A connected, undirected, weighted graph $G = (V, E)$

Output: A minimum spanning tree T

```

1 Choose an arbitrary vertex  $r \in V$ 
2  $T \leftarrow \emptyset$ 
3  $S \leftarrow \{r\}$ 
4 while  $S \neq V$  do
5   Select a minimum-weight edge  $(u, v)$  such that  $u \in S$  and  $v \in V \setminus S$ 
6   Add  $(u, v)$  to  $T$ 
7   Add  $v$  to  $S$ 
8 return  $T$ 

```

Algorithm 2 starts from an arbitrary vertex r and grows the spanning tree greedily. At each step, it selects the minimum-weight edge connecting a vertex already included in the tree to a vertex not yet included. The selected edge and the newly reached vertex are then added to the tree. The algorithm terminates when all vertices are included, at which point $|T| = |V| - 1$ and T is a spanning tree.

Worked example.

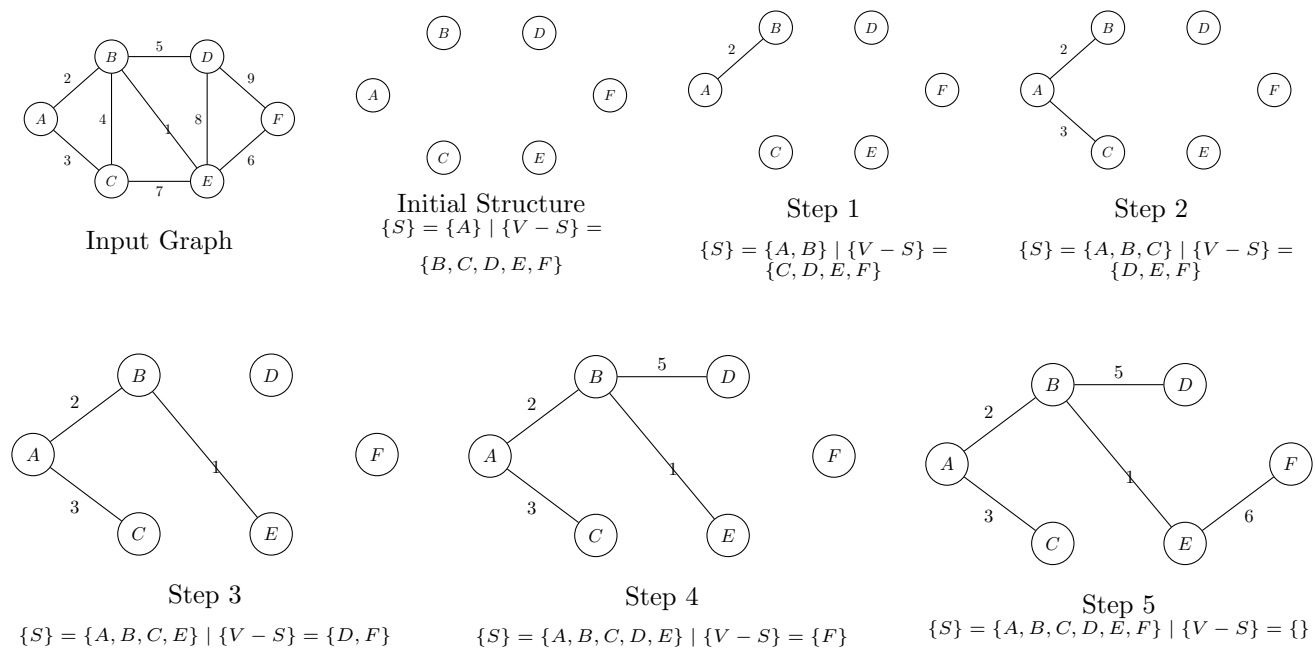


Figure 3: Worked example of Prim’s-Jarník Algorithm

The selected edges are AB , AC , BE , BD and EF . Thus, the resulting minimum spanning tree is $T = \{AB, AC, BE, BD, EF\}$ with total weight $w(T) = 2 + 3 + 1 + 5 + 6 = 17$.

4 Proof of Correctness of Prim’s-Jarník Algorithm

Claim 1 (Correctness). Prim’s-Jarník Algorithm returns a minimum spanning tree of the input graph.

Proof. We prove the correctness using the **cut property**. A cut is a partition of the vertex set V into two disjoint sets S and $V - S$. The edges having one endpoint in S and the other endpoint in $V - S$ are called the edges crossing the cut. The cut property states that, for any cut of a weighted graph, a minimum-weight edge crossing the cut belongs to some minimum spanning tree. If all edge weights are distinct, the minimum-weight crossing edge is unique and belongs to every minimum spanning tree.

At any intermediate stage of the algorithm, let S be the set of vertices already included in the tree and let $V - S$ be the set of vertices not yet included. The algorithm selects the minimum-weight edge $e^* = (u, v)$ such that

$u \in S$ and $v \in V - S$. Thus, e^* is the minimum-weight edge crossing the cut $(S, V - S)$.

Suppose, for contradiction, that e^* does not belong to an MST T' . Since T' is a spanning tree, there must be a path between u and v in T' . Because $u \in S$ and $v \in V - S$, this path must contain some edge e' that crosses the cut $(S, V - S)$.

Since e^* is the unique minimum-weight edge crossing this cut, $w(e^*) < w(e')$.

Now remove e' from T' . This disconnects T' into two components, with u in one component and v in the other. Adding e^* reconnects the two components and produces another spanning tree, say T'' .

Its total weight is

$$w(T'') = w(T') - w(e') + w(e^*).$$

Since $w(e^*) < w(e')$,

$$w(T'') < w(T').$$

This contradicts the assumption that T' is a minimum spanning tree.

Therefore, e^* must belong to an MST. Hence, every edge selected by Prim's-Jarník Algorithm is safe.

The algorithm continues selecting such edges until all vertices are included. Since every selected edge connects a new vertex to the existing tree, no cycle is created. When all vertices are included, exactly $|V| - 1$ edges have been selected. Therefore, the resulting tree is a minimum spanning tree.

Hence, Prim's-Jarník Algorithm is correct. $\square$

Time Complexity Analysis. The algorithm starts by choosing an arbitrary vertex and adding it to S , which takes $O(1)$ time. In each iteration, it searches all E edges to find the minimum-weight edge with one endpoint in S and the other in $V - S$, taking $O(E)$ time. Since each iteration adds exactly one new vertex and the algorithm terminates after $|V| - 1$ iterations, the total time is

$$O((|V| - 1)|E|) = O(|V||E|).$$

Adding the selected edge and vertex takes $O(1)$ time per iteration, contributing $O(|V|)$ overall, which is dominated by $O(|V||E|)$. Hence, the overall running time is

$$\boxed{O(|V||E|)}.$$

Reference. Kleinberg, J., & Tardos, E. (2006). *Algorithm Design*, Chapter 1. Pearson/Addison-Wesley.